MPI Sorting Performance Evaluation

Graphs and discussion for Ranking, Bucket, Odd-Even, Shell, and Bitonic sort

Student name	Group
Hermeneanu Ionut Silviu	10LF232

1. Evaluation method

The discussion below uses the timing data from *MPI_Sorting_Analysis.xlsx*. For each algorithm, the evaluation uses **Total_Time, Comm_Time, Comp_Time**, and **Speedup = T1 / Tp**. Communication overhead is interpreted with **Comm_Overhead_Pct = Comm_Time / Total_Time × 100**.

- Total time = end-to-end runtime for one MPI execution.
- Communication time = MPI data exchange and synchronization cost.
- Computation time = local sorting / processing cost.
- Speedup compares the 1-process baseline against p processes for the same algorithm and vector size.

2. Overall summary

Algorithm	Best measured speedup	Main takeaway
Bucket	2.19x at 4 proc. (N=50,000,000)	Best on medium/large inputs, but small inputs are communication-bound.
Odd-Even	1.01x at 2 proc. (N=100,000)	Scales poorly because repeated neighbor exchanges dominate.
Shell	3.57x at 4 proc. (N=50,000,000)	Best practical scalability and low communication cost.
Bitonic	1.00x at 1 proc. (N=100,000)	Structured but slower than the 1-process baseline in these runs.
Ranking	7.11x at 10 proc. (N=100,000)	Good speedup, but $O(n^2)$ cost keeps absolute runtime very high.

Interpretation: the best speedup and the best absolute runtime are not always the same thing. Ranking sort scales numerically, but Shell sort is the strongest overall practical performer in this dataset because it combines useful speedup with consistently low communication overhead.

3. Bucket sort

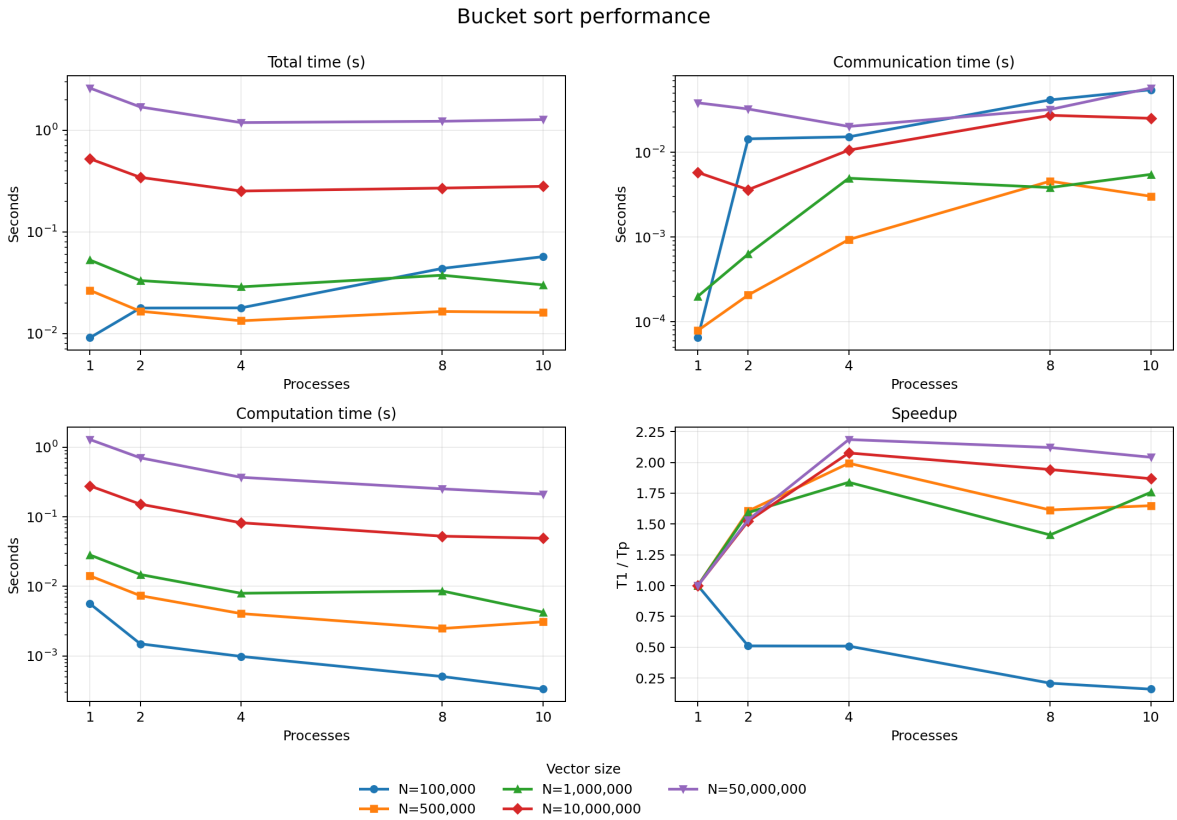

Figure 1. Bucket sort: total time, communication time, computation time, and speedup versus process count.

- Bucket sort scales reasonably well once the input is large enough to amortize communication.
- For 500,000 to 50,000,000 elements, the best timings are generally obtained at 4 processes, with speedup between about 1.84x and 2.19x.
- For the smallest case (100,000 elements), the parallel version is slower than the 1-process baseline because communication dominates; the communication share rises above 80% at 2–10 processes.
- On larger inputs, communication overhead drops to a much smaller share, which is why bucket sort starts to benefit from parallelism.
- The main overhead is bucket redistribution: values must be exchanged so that each process owns the correct value range.

4. Odd-Even sort

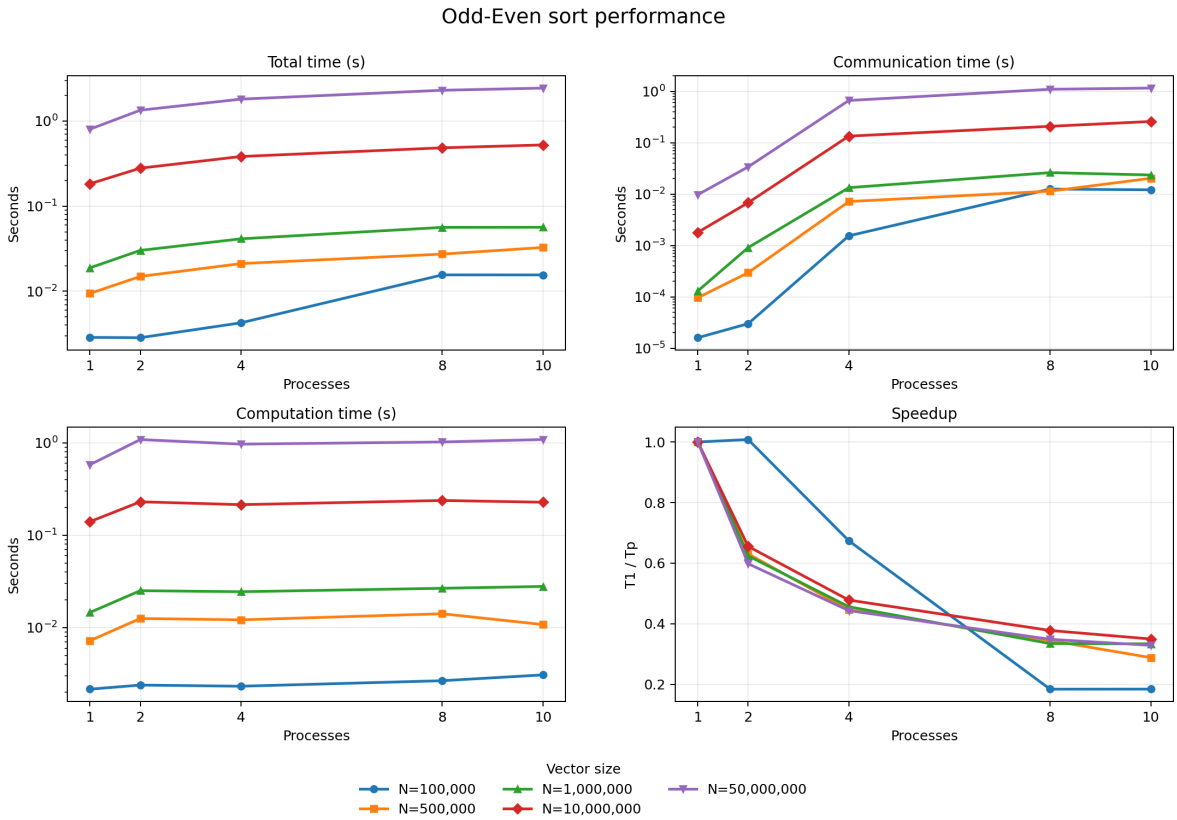

Figure 2. Odd-Even sort: total time, communication time, computation time, and speedup versus process count.

- Odd-even sort shows very poor scalability in these MPI measurements.
- The only slight gain appears at 100,000 elements with 2 processes (~1.01x speedup); all other parallel runs are slower than the 1-process baseline.
- From 4 processes upward, communication and synchronization overhead dominate, often consuming roughly one-third to four-fifths of total runtime.
- This is expected because odd-even transposition requires many repeated phase-by-phase neighbor exchanges and synchronization points.
- In practice, the algorithm is easy to implement in MPI, but it is not competitive here for performance.

5. Shell sort

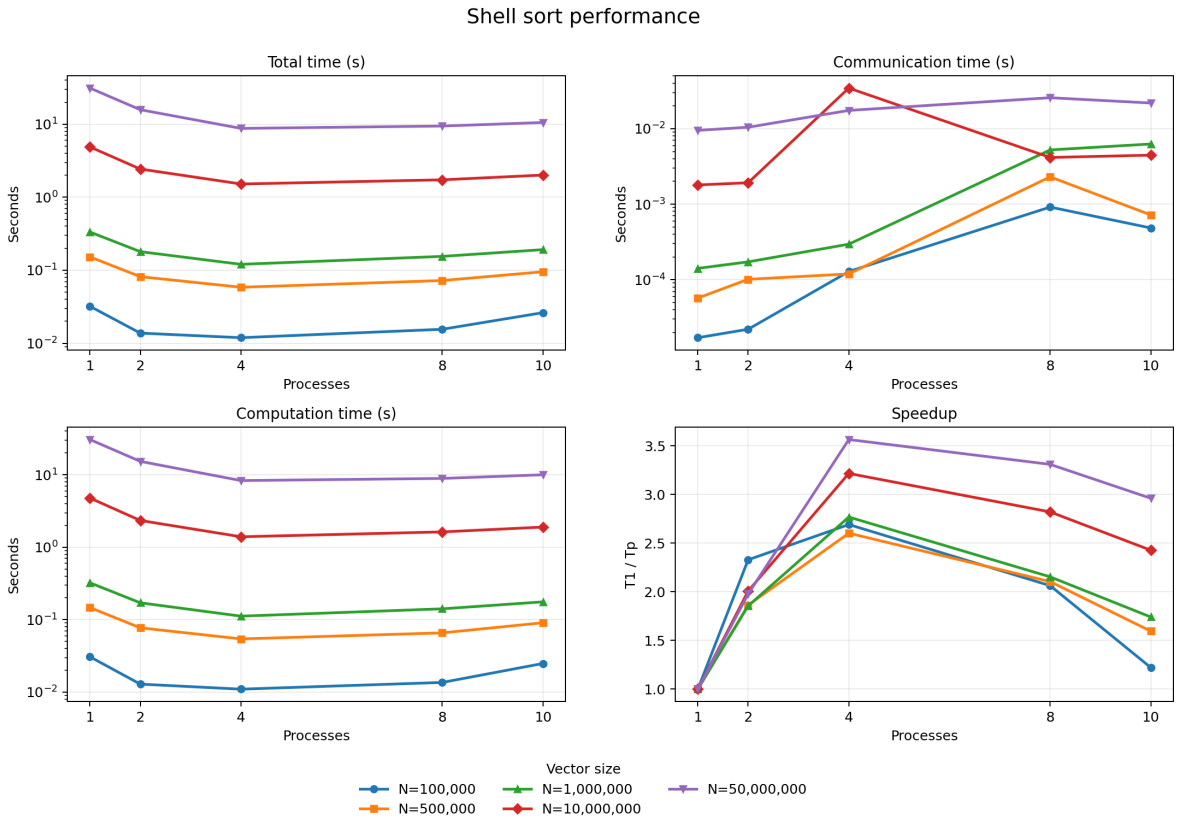

Figure 3. Shell sort: total time, communication time, computation time, and speedup versus process count.

- Shell sort is the best practical parallel performer in this dataset.
- Its best timings are consistently reached at 4 processes for every tested size, with speedup around 2.60x to 3.57x.
- Communication overhead stays very low in most cases, so runtime remains largely computation-dominated.
- At 8 and 10 processes the speedup starts to flatten or decline, which indicates diminishing returns and extra coordination overhead.
- Overall, Shell sort gives the best balance between lower total time, useful speedup, and small communication cost.

6. Bitonic sort

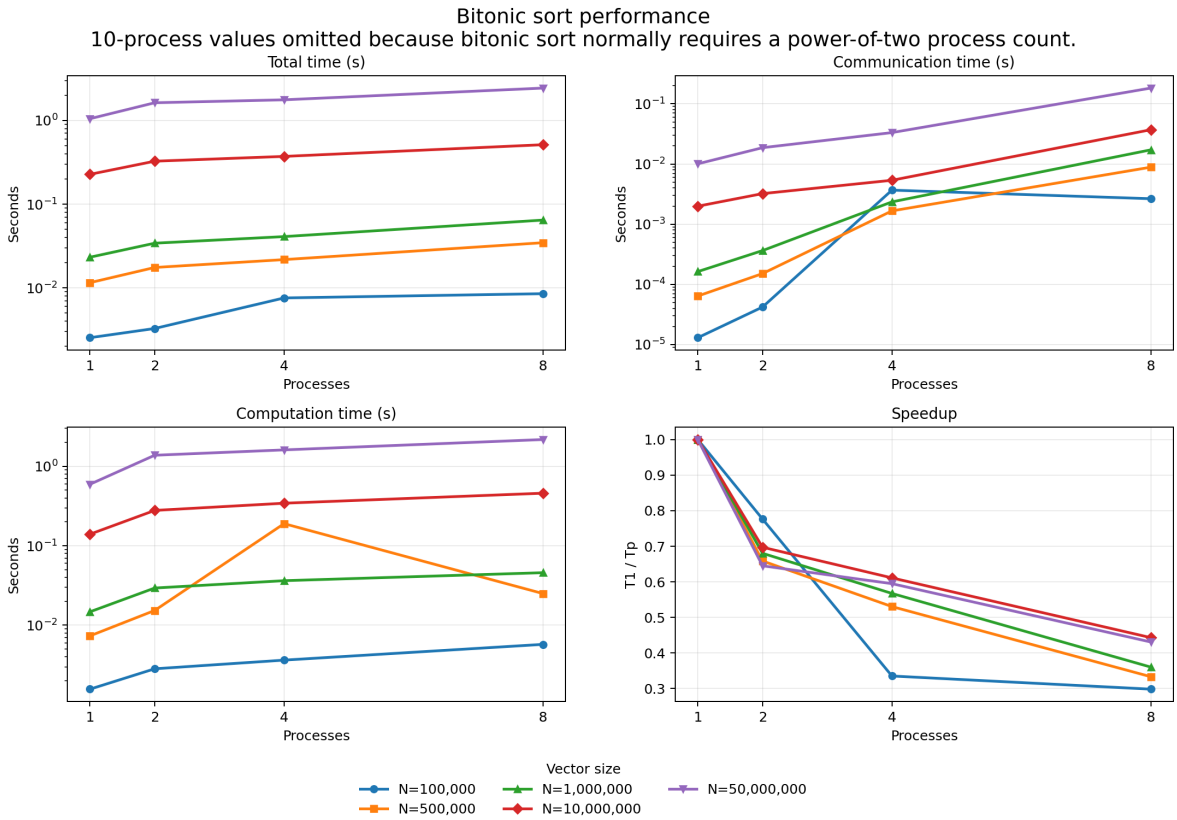

Figure 4. Bitonic sort: total time, communication time, computation time, and speedup versus process count.

- Bitonic sort does not show useful speedup for the valid process counts in this dataset; the 1-process baseline remains the fastest line for every tested input size.
- Although communication overhead becomes smaller for larger arrays, the repeated compare-exchange stages still add enough work that the parallel version does not beat the baseline.
- For smaller cases, communication becomes especially costly; for example, at 100,000 elements the communication share is about 48% at 4 processes and about 31% at 8 processes.
- The reported 10-process Bitonic points should not be interpreted: classical bitonic sort normally assumes a power-of-two number of processes, so the near-zero timings are a measurement artifact rather than a real speedup.

Important note: the 10-process bitonic values were excluded from interpretation because bitonic sort normally expects a power-of-two process count.

7. Ranking sort

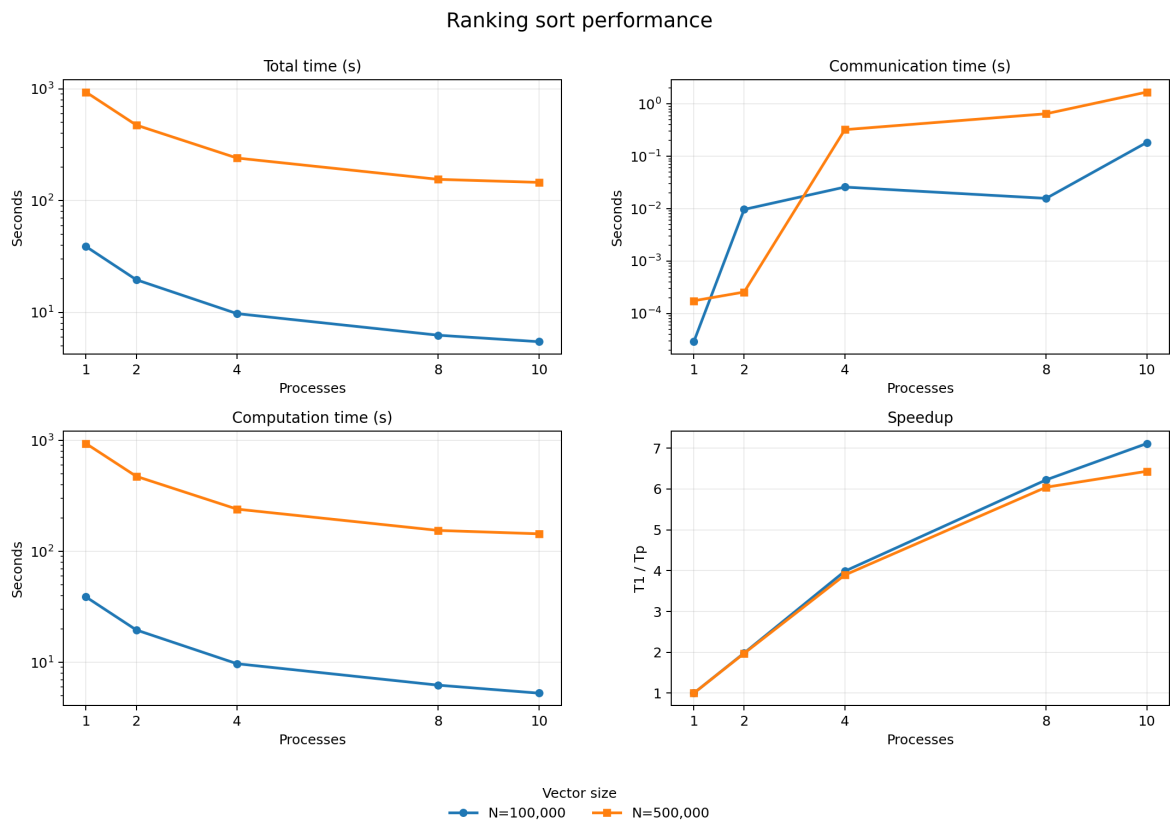

Figure 5. Ranking sort: total time, communication time, computation time, and speedup versus process count.

- Ranking sort shows strong parallel speedup on the sizes that were tested, reaching about 7.11x at 100,000 elements and about 6.43x at 500,000 elements.
- Communication overhead stays very low, so the parallel behavior is mainly limited by computation rather than MPI communication.
- However, the absolute runtime is still extremely high: about 38.9 s for 100,000 elements on 1 process and about 937.4 s for 500,000 elements on 1 process.
- This is the clearest example of why asymptotic complexity matters: ranking sort parallelizes, but its $O(n^2)$ work makes it impractical for the largest vectors.
- That is why the workbook only contains ranking data up to 500,000 elements.

8. Comparative discussion and conclusion

- Best overall practical result: Shell sort is the strongest parallel choice in this dataset. It provides the highest useful speedups on the full range of tested sizes and keeps communication overhead small.
- Best moderate scaling: Bucket sort becomes worthwhile for medium and large vectors, but it is sensitive to redistribution overhead, especially on small inputs.
- Weak MPI scalability: Odd-even sort spends too much time in repeated neighbor exchanges and synchronization, so its speedup is poor or negative for almost all multi-process runs.
- Structured but not efficient here: Bitonic sort has a regular communication pattern, but the added compare-exchange stages outweigh the benefits in these measurements.
- Parallel but still impractical: Ranking sort demonstrates that good speedup does not automatically imply good absolute performance. Even after parallelization, its runtime remains much larger than the other methods.

- Recommended conclusion for the assignment: the best MPI sorting algorithm is the one that balances local computation with modest communication. In this dataset, that balance is achieved most clearly by Shell sort, followed by Bucket sort for larger inputs.

Reusable paragraph for your submission

The experimental results show that MPI performance depends not only on the amount of parallel work, but also on communication and synchronization cost. Shell sort delivered the best practical scalability in the collected data, while bucket sort also performed well on larger vectors once communication overhead was amortized. Odd-even and bitonic sort were less effective in this environment because their parallel execution required more coordination between processes. Ranking sort achieved strong speedup for the tested cases, but its quadratic complexity made it impractical for the largest inputs. Overall, the measurements confirm that an MPI sorting algorithm is successful when it reduces computation time without introducing too much communication overhead.